Để làm được dạng đề này, sinh viên chỉ cần nắm vài mảng kiến thức cốt lõi của học phần: kết nối Node.js với DynamoDB, render dữ liệu ra EJS/HTML, viết CRUD API, tìm kiếm, kiểm tra dữ liệu nhập, và upload ảnh lên S3. Đây cũng đúng với trọng tâm môn CNMTPTUD: xây dựng web app Node.js kết nối DynamoDB và thao tác dữ liệu lớn trên AWS.

1) Kiến thức cần biết

a) Cấu trúc một web app Node.js cơ bản

Cần hiểu:

- `app.js` hoặc `server.js`
- routes
- controllers
- views EJS
- thư mục `public`
- file cấu hình AWS / biến môi trường

Ví dụ cấu trúc:

```
project/
├── app.js
├── routes/
│   └── ticketRoutes.js
├── controllers/
│   └── ticketController.js
├── views/
│   ├── index.ejs
│   ├── add.ejs
│   └── edit.ejs
├── public/
│   └── css/style.css
├── config/
│   └── aws.js
├── uploads/
├── .env
└── package.json
```

b) Express + EJS

Sinh viên cần biết:

- tạo server Express
- cấu hình `view engine = ejs`
- nhận dữ liệu form bằng `express.urlencoded()`
- phục vụ CSS bằng `express.static()`

Snippet:

```
const express = require("express");
const path = require("path");
const app = express();

app.set("view engine", "ejs");
app.set("views", path.join(__dirname, "views"));

app.use(express.urlencoded({ extended: true }));
app.use(express.json());
app.use(express.static("public"));
```

c) DynamoDB

Theo đề mẫu, sinh viên cần biết tạo bảng, giải thích khóa, rồi thao tác CRUD và query/scan dữ liệu.

Cần hiểu:

- Table name
- Partition key
- kiểu dữ liệu String / Number
- `PutCommand`, `GetCommand`, `ScanCommand`, `UpdateCommand`, `DeleteCommand`

Snippet cấu hình:

```
const { DynamoDBClient } = require("@aws-sdk/client-dynamodb");
const { DynamoDBDocumentClient } = require("@aws-sdk/lib-dynamodb");

const client = new DynamoDBClient({
  region: process.env.AWS_REGION,
  credentials: {
    accessKeyId: process.env.AWS_ACCESS_KEY_ID,
    secretAccessKey: process.env.AWS_SECRET_ACCESS_KEY
  }
});

const docClient = DynamoDBDocumentClient.from(client);

module.exports = docClient;
```

d) CRUD cơ bản

Ví dụ thêm dữ liệu:

```
const { PutCommand } = require("@aws-sdk/lib-dynamodb");
const docClient = require("../config/aws");

async function createTicket(ticket) {
  const params = {
    TableName: "EventTickets",
    Item: ticket
  };
```

```
  await docClient.send(new PutCommand(params));
}
```

Đọc danh sách:

```
const { ScanCommand } = require("@aws-sdk/lib-dynamodb");

async function getAllTickets() {
  const params = { TableName: "EventTickets" };
  const result = await docClient.send(new ScanCommand(params));
  return result.Items || [];
}
```

Xóa:

```
const { DeleteCommand } = require("@aws-sdk/lib-dynamodb");

async function deleteTicket(ticketId) {
  const params = {
    TableName: "EventTickets",
    Key: { ticketId }
  };
  await docClient.send(new DeleteCommand(params));
}
```

Cập nhật:

```
const { UpdateCommand } = require("@aws-sdk/lib-dynamodb");

async function updateTicket(ticketId, data) {
  const params = {
    TableName: "EventTickets",
    Key: { ticketId },
    UpdateExpression: `
      SET eventName = :eventName,
          price = :price,
          quantity = :quantity
    `,
    ExpressionAttributeValues: {
      ":eventName": data.eventName,
      ":price": Number(data.price),
      ":quantity": Number(data.quantity)
    }
  };
  await docClient.send(new UpdateCommand(params));
}
```

e) Tìm kiếm

Đề kiểu này thường cần tìm gần đúng theo tên hoặc lọc theo trạng thái. Sinh viên chỉ cần nhớ `ScanCommand` + `FilterExpression`.

Ví dụ:

```
const { ScanCommand } = require("@aws-sdk/lib-dynamodb");

async function searchTickets(keyword) {
  const params = {
    TableName: "EventTickets",
    FilterExpression: "contains(eventName, :kw)",
    ExpressionAttributeValues: {
      ":kw": keyword
    }
  };

  const result = await docClient.send(new ScanCommand(params));
  return result.Items || [];
}
```

f) Validation dữ liệu nhập

Đây là phần rất hay bị mất điểm trong Câu 3 nếu chỉ chạy được mà không ràng buộc dữ liệu. Đề mẫu cũng chấm nặng phần API thêm "đầy đủ ràng buộc".

Ví dụ:

```
function validateTicket(data) {
  const errors = [];

  if (!data.ticketId || data.ticketId.trim() === "") {
    errors.push("Mã vé không được rỗng");
  }

  if (!data.eventName || data.eventName.trim().length < 3) {
    errors.push("Tên sự kiện phải có ít nhất 3 ký tự");
  }

  if (isNaN(data.price) || Number(data.price) <= 0) {
    errors.push("Giá vé phải > 0");
  }

  if (isNaN(data.quantity) || Number(data.quantity) < 0) {
    errors.push("Số lượng phải >= 0");
  }

  return errors;
}
```

Dùng trong controller:

```
const errors = validateTicket(req.body);
if (errors.length > 0) {
  return res.render("add", { errors, oldData: req.body });
}
```

g) Upload ảnh lên S3

AWS S3, sinh viên nên biết:

- nhận file từ form
- dùng `multer` để đọc file upload
- dùng `S3Client` + `PutObjectCommand` để đẩy file lên bucket

Snippet:

```
const multer = require("multer");
const upload = multer({ storage: multer.memoryStorage() });

module.exports = upload;
```

Route:

```
const express = require("express");
const router = express.Router();
const upload = require("../middlewares/upload");
const controller = require("../controllers/ticketController");

router.post("/tickets/add", upload.single("image"), controller.createTicket);
```

Cấu hình S3:

```
const { S3Client, PutObjectCommand } = require("@aws-sdk/client-s3");

const s3 = new S3Client({
  region: process.env.AWS_REGION,
  credentials: {
    accessKeyId: process.env.AWS_ACCESS_KEY_ID,
    secretAccessKey: process.env.AWS_SECRET_ACCESS_KEY
  }
});

async function uploadToS3(file) {
  const fileName = Date.now() + "-" + file.originalname;

  const command = new PutObjectCommand({
    Bucket: process.env.S3_BUCKET_NAME,
    Key: fileName,
    Body: file.buffer,
    ContentType: file.mimetype
  });

  await s3.send(command);

  return
`https://${process.env.S3_BUCKET_NAME}.s3.${process.env.AWS_REGION}.amazonaws
.com/${fileName}`;
}
```

h) Logic tính toán nghiệp vụ

Phần này giúp Câu 3 khó hơn. Thường chỉ cần một công thức nghiệp vụ đơn giản.

Ví dụ:

```
function calculateTotalRevenue(price, quantitySold) {
  return Number(price) * Number(quantitySold);
}

function calculateRemaining(quantity, quantitySold) {
  return Number(quantity) - Number(quantitySold);
}
```

Hoặc phân loại:

```
function getTicketStatus(quantity) {
  if (quantity <= 0) return "Hết vé";
  if (quantity <= 10) return "Sắp hết";
  return "Còn vé";
}
```

2) Thư viện nên cài

Các thư viện đủ dùng cho dạng đề này:

```
npm init -y
npm install express ejs dotenv multer
npm install @aws-sdk/client-dynamodb @aws-sdk/lib-dynamodb
npm install @aws-sdk/client-s3
```

Nếu muốn tự sinh id:

```
npm install uuid
```

Ví dụ:

```
const { v4: uuidv4 } = require("uuid");
const ticketId = uuidv4();
```

3) Một form EJS mẫu

```
<form action="/tickets/add" method="POST" enctype="multipart/form-data">
  <input type="text" name="ticketId" placeholder="Mã vé">
  <input type="text" name="eventName" placeholder="Tên sự kiện">
  <input type="number" name="price" placeholder="Giá vé">
  <input type="number" name="quantity" placeholder="Số lượng">
  <input type="file" name="image">
  <button type="submit">Thêm vé</button>
</form>
```

Render danh sách:

```
<table border="1">
  <tr>
    <th>Mã vé</th>
    <th>Sự kiện</th>
    <th>Giá</th>
    <th>Số lượng</th>
    <th>Ảnh</th>
  </tr>

  <% tickets.forEach(ticket => { %>
    <tr>
      <td><%= ticket.ticketId %></td>
      <td><%= ticket.eventName %></td>
      <td><%= ticket.price %></td>
      <td><%= ticket.quantity %></td>
      <td>
        <% if (ticket.imageUrl) { %>
          <img src="<%= ticket.imageUrl %>" width="80">
        <% } %>
      </td>
    </tr>
  <% }) %>
</table>
```

4) Những lỗi sinh viên hay gặp

- quên `express.urlencoded(...)` nên form không nhận dữ liệu
- tên field trong form khác tên field trong controller
- khóa chính DynamoDB không đúng với `Key`
- `price`, `quantity` không ép kiểu số
- quên `enctype="multipart/form-data"` khi upload file
- upload S3 thành công nhưng không lưu `imageUrl` vào DynamoDB
- validation có viết nhưng không chặn lưu dữ liệu sai
- tìm kiếm dùng `contains(...)` nhưng sai tên thuộc tính

5) Cách ôn ngắn gọn trước khi làm bài

Sinh viên chỉ cần luyện theo thứ tự này:

1. Tạo project Express + EJS
2. Kết nối DynamoDB
3. Hiển thị danh sách dữ liệu
4. Viết thêm, sửa, xóa
5. Viết tìm kiếm
6. Thêm validation
7. Upload ảnh lên S3
8. Thêm 1 hàm tính toán nghiệp vụ